Reproducing "Score-Based Generative Modeling Through Stochastic Differential Equations"

Jonathan Tadesse
jtadesse@kth.se

Abstract

This project studies score-based diffusion models for image generation via stochastic differential equations (SDEs), based on the framework of Song et al. [4]. We compare variance-preserving (VP) and variance-exploding (VE) SDE formulations using U-Net, NCSN++, and DDPM++ architectures. Model performance is evaluated using Fréchet Inception Distance (FID), while memorization is assessed via nearest-neighbor mean squared error (NN MSE) and train–test gap metrics. We further analyze the effect of data augmentation on both sample quality and memorization behavior.

1 Introduction

Diffusion models have become an important class of generative models due to their high-quality image synthesis via reversing a gradual noising process. In this project, we reproduce parts of Song et al. [4], which formulates image generation as reversing a stochastic noising process using SDEs.

A full reproduction of the paper was not feasible within the scope of this project, as the original models require large datasets and extensive sampling. Instead, we focus on a smaller, controlled setting using the CIFAR-10 bird class as the main dataset, and Oxford Flowers as a secondary dataset to evaluate generalization across domains.

Our goal is to reproduce the core idea of score-based SDE generation in a smaller setting rather than match state-of-the-art image quality. We compare VP-SDE and VE-SDE models using U-Net, NCSN++, and DDPM++ architectures, evaluating them with FID and nearest-neighbor memorization metrics. We also study the effect of data augmentation on sample quality and overfitting.

Our experiments show that the models learned clear image structure, but the generated images were still far from state-of-the-art quality. The best FID values were obtained by non-augmented models, but these also showed stronger signs of memorization. Overall, augmentation helped reduce memorization, but did not consistently improve FID.

2 Theory

2.1 Score-based SDE framework

The goal of a generative model is to learn a data distribution $p_{\text{data}}(x)$ such that new samples can be generated from it. Score-based generative modeling does this by learning the *score* at each intermediate time step,:

$$\nabla_x \log p_t(x),$$

which is a function that points toward regions of higher density in the perturbed data distribution.

Song et al. [4] formulate diffusion and score-based models using a continuous-time stochastic differential equation (SDE). The forward process gradually corrupts clean data $x(0) \sim p_{\text{data}}$ into noise:

$$dx = f(x,t)\,dt + g(t)\,dW_t, \tag{1}$$

where $f(x,t)$ is the drift, $g(t)$ is the diffusion coefficient, and W_t is a standard Wiener process. The choice of f and g determines how the image is noised over time.

Sampling is done by running the corresponding reverse-time SDE from noise back to data:

$$dx = \left[f(x,t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t)\, d\bar{W}_t, \tag{2}$$

where $d\bar{W}_t$ denotes a reverse-time Wiener process. Since the true score $\nabla_x \log p_t(x)$ is unknown, it is approximated by a neural network $s_\theta(x,t)$, which is trained with denoising score matching by minimizing:

$$\mathcal{L}(\theta) = \mathbb{E}_{t\sim U(0,T)} \left[\lambda(t) \mathbb{E}_{x(0)} \mathbb{E}_{x(t)|x(0)} \left\| s_\theta(x(t),t) - \nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) \right\|_2^2\right]. \tag{3}$$

Here, $p_{0t}(x(t) \mid x(0))$ is the perturbation kernel of the forward SDE and $\lambda(t)$ is a weighting function. Once trained, s_θ can be used in Eq. 2 to generate samples.

2.2 VP-SDE and VE-SDE

We use two SDE families from Song et al. [4]. The variance preserving SDE (VP-SDE) is closely related to DDPMs and gradually reduces the signal while adding Gaussian noise:

$$dx = -\frac{1}{2}\beta(t) x\, dt + \sqrt{\beta(t)}\, dW. \tag{4}$$

We use the linear schedule $\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$, with $\beta_{\min} = 0.1$ and $\beta_{\max} = 20.0$.

The variance exploding SDE (VE-SDE) instead keeps the drift zero and increases the noise scale over time:

$$dx = \sqrt{\frac{d\sigma^2(t)}{dt}}\, dW. \tag{5}$$

For VE-SDE, we use the exponential noise schedule

$$\sigma(t) = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^t, \text{ with } \sigma_{\min} = 0.01, \text{ and } \sigma_{\max} = 50.0.$$

Both SDEs have Gaussian perturbation kernels $p_{0t}(x(t) \mid x(0))$. Therefore, a noisy image can be written as

$$x(t) = m(t)x(0) + \mathrm{std}(t) z, \qquad z \sim \mathcal{N}(0, I),$$

where $m(t)$ and $\mathrm{std}(t)$ are determined by the chosen SDE. This is the form used in the training loss.

2.3 Loss function

Since the perturbation kernels are Gaussian, the target score can be written as

$$\nabla_{x(t)} \log p_{0t}(x(t) \mid x(0)) = -\frac{z}{\mathrm{std}(t)}.$$

In implementation, we use the weighted loss

$$\mathcal{L}(\theta) = \mathbb{E}_{t,x(0),z} \left[\mathrm{std}^2(t) \left\| s_\theta(x(t),t) + \frac{z}{\mathrm{std}(t)} \right\|_2^2\right], \tag{6}$$

which corresponds to Eq. 3 with $\lambda(t) = \mathrm{std}^2(t)$. This trains the network to predict the score at different noise levels.

2.4 Sampling

In practice, the reverse SDE must be discretized. We use the predictor-corrector sampling framework from Song et al. [4]. The predictor step applies an Euler–Maruyama update to move the sample backward in time. The optional corrector step refines the sample at the current noise level using Langevin dynamics:

$$x \leftarrow x + \epsilon s_\theta(x,t) + \sqrt{2\epsilon}\, z, \qquad z \sim \mathcal{N}(0, I), \tag{7}$$

where ϵ is the step size. See Appendix 1 for a more detailed explanation of the PC algorithm.

Corrector steps can improve sample quality, but they also increase sampling time. In our experiments, we used 1000 predictor steps.

2.5 FID and memorization

Fréchet Inception Distance (FID) measures the distance between real and generated images in the feature space of a pre-trained Inception network. In our implementation, we calculated FID using the PyTorch `torchmetrics.image.fid.FrechetInceptionDistance` library with 2048 dimensional inception features. Before computing FID, generated and real images are converted from the normalized range $[-1, 1]$ back to unsigned 8-bit image format. If the real and generated feature distributions have empirical means and covariances (μ_r, Σ_r) and (μ_g, Σ_g), then

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right). \tag{8}$$

Lower FID means that the generated distribution is closer to the real distribution in this feature space.

To compare train and test closeness, we use the FID gap

$$\Delta_{\mathrm{FID}} = \mathrm{FID}_{\mathrm{test}} - \mathrm{FID}_{\mathrm{train}}. \tag{9}$$

A positive value means that generated samples are closer to the training set than to the test set in Inception feature space.

FID does not directly show whether individual training images were copied. Therefore, we also use nearest-neighbor pixel MSE. For a generated image $\hat{x}$, we define

$$d_{\mathrm{train}}(\hat{x}) = \min_{x_i \in \mathcal{D}_{\mathrm{train}}} \frac{1}{CHW}\|\hat{x} - x_i\|_2^2, \qquad d_{\mathrm{test}}(\hat{x}) = \min_{x_i \in \mathcal{D}_{\mathrm{test}}} \frac{1}{CHW}\|\hat{x} - x_i\|_2^2. \tag{10}$$

The nearest-neighbor gap is

$$\Delta_{\mathrm{NN}} = \mathbb{E}_{\hat{x}}[d_{\mathrm{test}}(\hat{x})] - \mathbb{E}_{\hat{x}}[d_{\mathrm{train}}(\hat{x})]. \tag{11}$$

A positive Δ_{NN} means that generated images are closer to training images than to test images on average. We use Δ_{FID} and Δ_{NN} as diagnostic indicators of overfitting, not as proof of memorization by themselves.

2.6 Exponential Moving Average (EMA)

Exponential moving average (EMA) is used to obtain a smoother version of the model parameters. We maintain an exponential moving average of the parameters instead of using only the latest θ:

$$\theta_{\mathrm{EMA}} \leftarrow \gamma\theta_{\mathrm{EMA}} + (1-\gamma)\theta,$$

where γ is the EMA decay and $\theta_{\mathrm{EMA}}^{(0)} = \theta^{(0)}$. EMA reduces the effect of noisy parameter updates and often gives more stable generated samples.

3 Related work

Diffusion models build on denoising, score matching, and probabilistic diffusion models. Sohl-Dickstein et al. [2] introduced diffusion probabilistic models, while Ho et al. [1] later showed that DDPMs can generate high-quality images, although sampling often requires many denoising steps. Song et al. [4] generalize this to continuous time with stochastic differential equations, connecting score-based models and DDPMs through VP-SDE and VE-SDE formulations. DDIM [3] addresses slow sampling with fewer denoising steps, which is relevant as a possible extension. Since the score-based SDE paper uses U-Net-style architectures such as NCSN++, we compare U-Net, NCSN++, and DDPM++ style networks under limited compute.

4 Data

For this project, we primarily use the CIFAR-10 dataset, which contains 60,000 RGB images of size 32×32 across 10 classes, split into 50,000 training and 10,000 test images. Due to computational constraints, we restrict the main experiments to the bird class, resulting in 5,000 training and 1,000 test images. As a secondary dataset, we use the Oxford Flowers dataset at 48×48 resolution. This dataset is not used for direct comparison to CIFAR-10, but to verify that the VP-SDE training and evaluation pipeline generalizes to a different natural-image distribution. In particular, it enables applying the same FID and memorization analysis to a separate domain, and additionally to study whether higher image resolution leads to improved FID scores.

5 Methods

5.1 Score-Based Training Objective

All models were trained using continuous denoising score matching. For each clean image $x_0 \sim p_{\text{data}}$, we sample a time $t \sim U(\epsilon, 1)$ and Gaussian noise $z \sim \mathcal{N}(0, I)$. The image is then perturbed using the VP-SDE or VE-SDE kernels defined in Section 2.2, as given in Eqs. 13 and 12.

$$x(t) = m(t)x_0 + \text{std}(t)z.$$

Here $m(t)$ and $\text{std}(t)$ are determined by the chosen SDE. The score network takes $x(t)$ and t as input and predicts

$$s_\theta(x(t), t) \approx \nabla_{x(t)} \log p_t(x(t)).$$

Since the perturbation kernels are Gaussian, the conditional target score is

$$\nabla_{x(t)} \log p_{0t}(x(t) \mid x_0) = -\frac{z}{\text{std}(t)}.$$

We therefore minimized the weighted score-matching loss from Eq. 6. In this way, the network learns to estimate the score of noisy images over a continuous range of noise levels.

5.2 Model architectures

We used three U-Net-based score-network architectures: NCSN++, DDPM++ and a U-Net baseline. All models take a noisy image $x(t)$ and a continuous time value t as input, and output a tensor with the same shape as the image, interpreted as the score estimate $s_\theta(x(t), t)$. The NCSN++ and DDPM++ models were adapted from the open-source `score_sde_pytorch` implementation, while the U-Net baseline was implemented separately.

NCSN++ was used for the VE-SDE experiments, where the model is conditioned on the noise scale $\sigma(t)$. DDPM++ and the U-Net baseline were used for the VP-SDE experiments, where the model is conditioned on the continuous time t. All architectures use time embeddings, encoder decoder skip connections, residual blocks, and self-attention at selected low resolutions. The attention layers were kept because they help model global image structure, which is important for CIFAR-10-scale generation. The exact model sizes are reported in Table 1 and the exact model configs are reported in Appendix 6.

5.3 Sampling procedure

After training, samples were generated by discretizing the reverse-time SDE in Eq. 2. For VP-SDE models, the initial samples were drawn from $x(1) \sim \mathcal{N}(0, I)$. For VE-SDE models, the initial samples were drawn from $x(1) \sim \mathcal{N}(0, \sigma_{\max}^2 I)$. The reverse process was simulated from $t = 1$ to $t = \epsilon$ using an Euler–Maruyama predictor step. We used 1000 predictor steps as [4] for the main evaluations. We also sampled one model VP-SDE U-net in isolation with $\{1, 2, 3, 4\}$ Langevin corrector steps to see the effect.

5.4 Evaluation protocol

We evaluated all models on sample quality and possible overfitting. Sample quality is measured using Fréchet Inception Distance (FID). We calculated the FID separately against the training and test sets. We then noted the train and test FID gap as outlined in section 2.5:

$$\Delta_{\text{FID}} = \text{FID}_{\text{test}} - \text{FID}_{\text{train}}$$

Since the Fréchet Inception Distance measures the similarity in image distribution between the generated image to the test/training distribution a positive Δ_{FID} would mean that the generated samples are closer to the training set than to the held out test set with respect to inception feature space. For the memorization analysis in Section 2.5, we report summary statistics of the distances and the nearest-neighbor gap Δ_{NN}. While these metrics alone do not establish memorization or overfitting, together they provide indicative evidence of such behavior.

6 Experiments

6.1 Experimental Setup

We converted the images to tensors and normalized to $[-1, 1]$. For CIFAR-10, we restricted the data to the bird class, giving 5000 training images and 1000 test images at 32×32 resolution. For Oxford

Flowers, images were resized to 48×48 and normalized in the same way, we used 1020 training images and 6198 test images. We then trained both non-augmented and augmented models. We augmented the CIFAR-10 data by applying horizontal flips, random crops with reflection padding. For the Oxford Flower data we used the same augmentation except we also used random affine transformations with rotations in the range $[-5°, 5°]$ and zoom factors in the range $[1.05, 1.3]$.

6.2 Compared Models

We compared three score-network architectures: NCSN++, DDPM++, and a U-Net baseline. NCSN++ was used for the VE-SDE experiments, while DDPM++ and the U-Net baseline were used for the VP-SDE experiments. The parameter counts are shown in Table 1; detailed architecture settings are given in Appendix 6.

Table 1: Model architectures and parameter counts.

Dataset	SDE	Architecture	Parameters (M)
CIFAR-10 Bird	VE-SDE	NCSN++	62.76
CIFAR-10 Bird	VP-SDE	DDPM++	10.09
CIFAR-10 Bird	VP-SDE	U-Net	62.14
Oxford Flowers	VP-SDE	DDPM++	9.89

6.3 Training Setup

Training was implemented in PyTorch using `torch` and `torchvision`. All main models were trained for 1760 epochs using AdamW with mini-batches of size 128, learning rate 10^{-4}, and weight decay 10^{-4}. The VP-SDE models used $\beta_{\min} = 0.1$ and $\beta_{\max} = 20.0$[4], while the VE-SDE models used $\sigma_{\min} = 0.01$ and $\sigma_{\max} = 50.0$ [4]. For all models, an exponential moving average of the parameters was maintained with decay $\gamma = 0.995$ and evaluated in addition to the raw network weights. For the reasons outlined in section 2.6

6.4 Evaluation Setup

For FID evaluation, generated images were obtained as described in Section 2.4. Samples were generated with 1000 predictor steps and no Langevin corrector steps. For CIFAR-10 Bird, FID was computed using 1000 generated samples, 1000 reference images from the plain bird training set and 1000 reference images from the plain bird test set. For Oxford Flowers, FID was computed using 1000 generated samples, 1000 training reference images and 1000 test reference images. For the memorization analysis, we generated 1000 samples per model and compared them to nearest neighbors from both the training and test sets using pixel-space MSE. We then reported summary statistics of the nearest-neighbor distances and the nearest-neighbor train test gap.

In addition to the main evaluation, we performed two isolated experiments on the non-augmented VP-SDE U-Net model. First, we generated 5000 samples and recomputed FID using the same train and test reference sets to study how the FID estimate changes with more generated samples, as shown in Figure 4. Second, we generated 1000 samples using Langevin corrector steps $c \in \{1, 2, 3, 4\}$, and repeated the same FID and nearest-neighbor calculations. These experiments are separate from the main comparison in Table 2.

7 Results

Table 2: FID and nearest-neighbour MSE results.

Dataset	**Architecture**	**SDE**	**Aug.**	**FID Train**	**FID Test**	**Δ FID**	**NN Train**	**NN Test**	**Δ NN**
CIFAR-10 Bird	NCSN++	VE	No	63.654	62.949	-0.705	0.102933	0.105205	0.002272
CIFAR-10 Bird	NCSN++	VE	Yes	69.430	68.876	-0.554	0.100371	0.101254	0.000883
CIFAR-10 Bird	DDPM++	VP	No	62.638	63.175	0.537	0.084804	0.087987	0.003182
CIFAR-10 Bird	DDPM++	VP	Yes	73.205	73.846	0.641	0.101829	0.102227	0.000397
CIFAR-10 Bird	U-Net	VP	No	57.910	59.301	1.391	0.056379	0.069794	0.013415
CIFAR-10 Bird	U-Net	VP	Yes	71.355	71.561	0.206	0.122441	0.124467	0.002026
Flower	DDPM++	VP	No	88.813	96.011	7.198	0.113438	0.132707	0.019270
Flower	DDPM++	VP	Yes	122.153	128.439	6.286	0.211084	0.198600	-0.012484

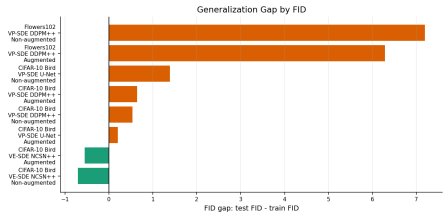


Figure 1: Generalization gap by FID.

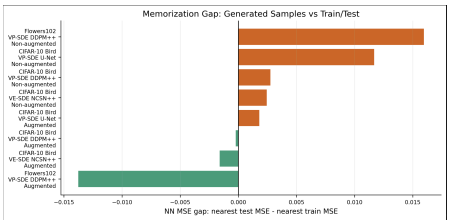


Figure 2: Memorization gap.

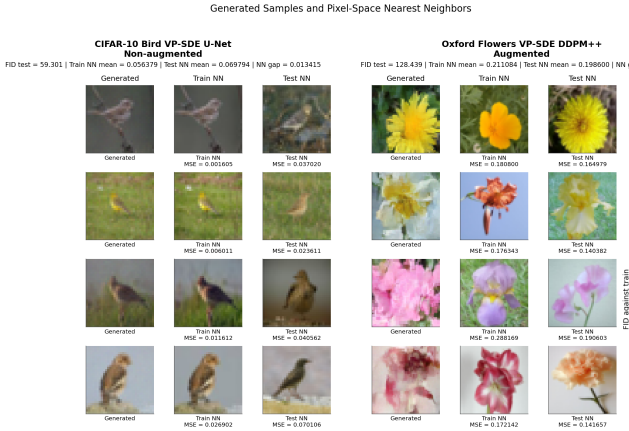


Figure 3: Best model by FID and best model by memorization: generated vs. train/test comparison. A larger version is shown in Appendix Figure A.1.

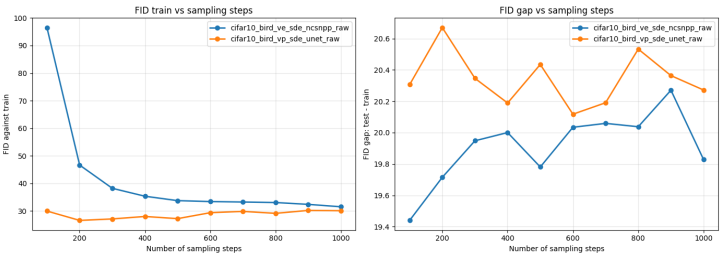


Figure 4: FID evolution using 5000 generated images for VP U-Net and VE NCSN++ trained on non-augmented data, instead of 1000 as in Table 2. A larger version is shown in Appendix Figure A.2.

Table 3: Effect of corrector steps on FID and nearest-neighbor memorization metrics for the non-augmented VP U-Net model. 1000 generated images were used in the FID calculations.

Corrector steps	FID train	FID test	FID gap	Train NN MSE	Test NN MSE	NN gap
0	**57.910**	**59.301**	1.391	0.056379	0.069794	0.013415
1	59.409	61.199	1.790	0.054426	0.066165	0.011739
2	58.920	60.641	1.722	0.053612	0.065503	0.011892
3	58.860	60.866	2.006	0.055193	0.066920	**0.011728**
4	59.108	60.324	**1.216**	0.054812	0.069601	0.014788

8 Conclusion

This project reproduced the main score-based SDE pipeline on CIFAR-10 Bird and Oxford Flowers under limited compute. Four model configurations were compared using FID, nearest-neighbor memorization metrics, and train-test overfitting indicators, with and without data augmentation. Data augmentation generally reduced memorization indicators, but often increased FID. The results also showed that using more generated samples improved the FID estimates for the two models in Figure 4, while adding more Langevin corrector steps did not improve results in our setting. Model capacity also appeared important: the large VP-SDE U-Net achieved the best CIFAR-10 Bird FID, but also showed the strongest memorization signal, suggesting that higher capacity can improve sample quality while increasing overfitting risk on small datasets. One possible reason for the limited corrector-step improvement is that the models were not trained long enough: we trained for 1760 epochs, whereas the original CIFAR-10 setup corresponds to roughly 3328 epochs of per-image exposure. Future work should therefore train for longer and compare architectures with similar parameter budgets to better separate the effects of SDE type, model size, and augmentation. Overall, the project improved our understanding of score-based diffusion models and why both image quality and memorization should be considered when evaluating generated samples.

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

[2] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics, 2015. URL https://arxiv.org/abs/1503.03585.

[3] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022. URL https://arxiv.org/abs/2010.02502.

[4] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations, 2021. URL https://arxiv.org/abs/2011.13456.

A Additional qualitative results

Large-Scale Visualizations of FID and Memorization

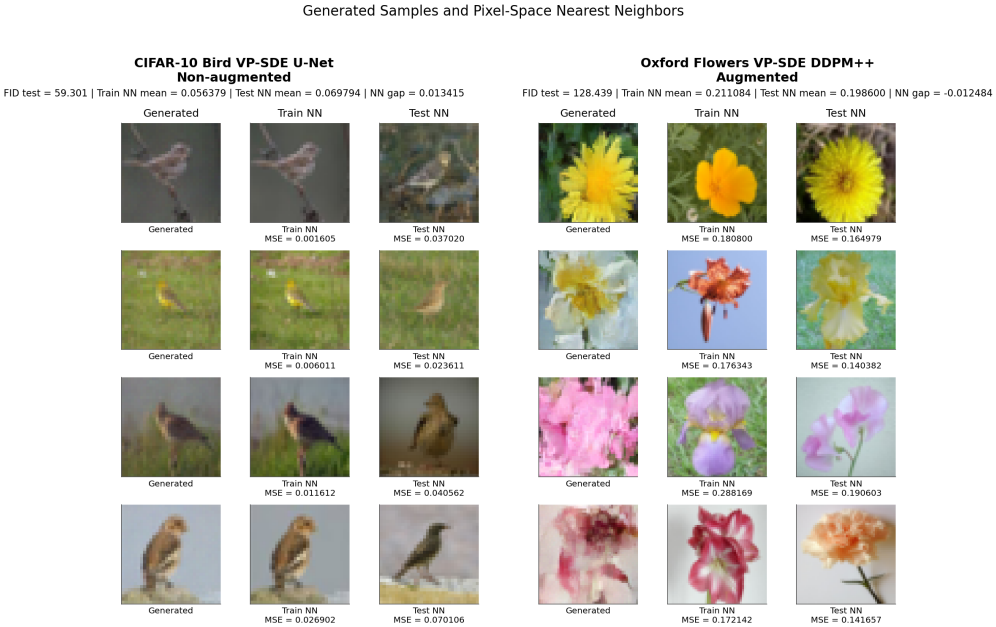


Figure A.1: Generated samples compared with nearest train and test images for the best FID model and the model with the strongest memorization signal.

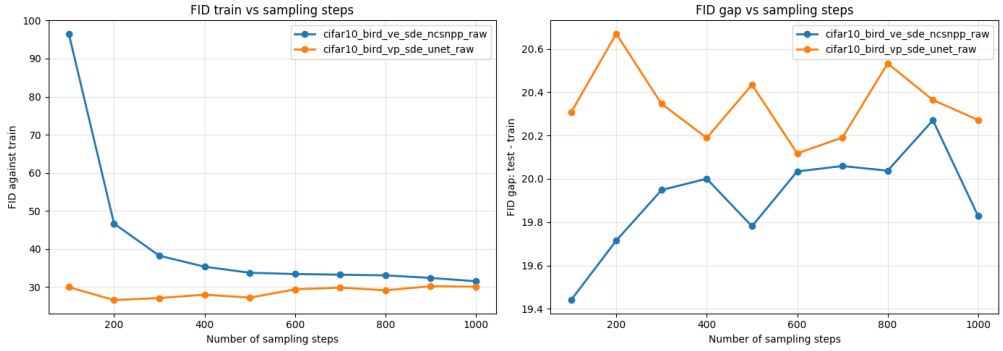
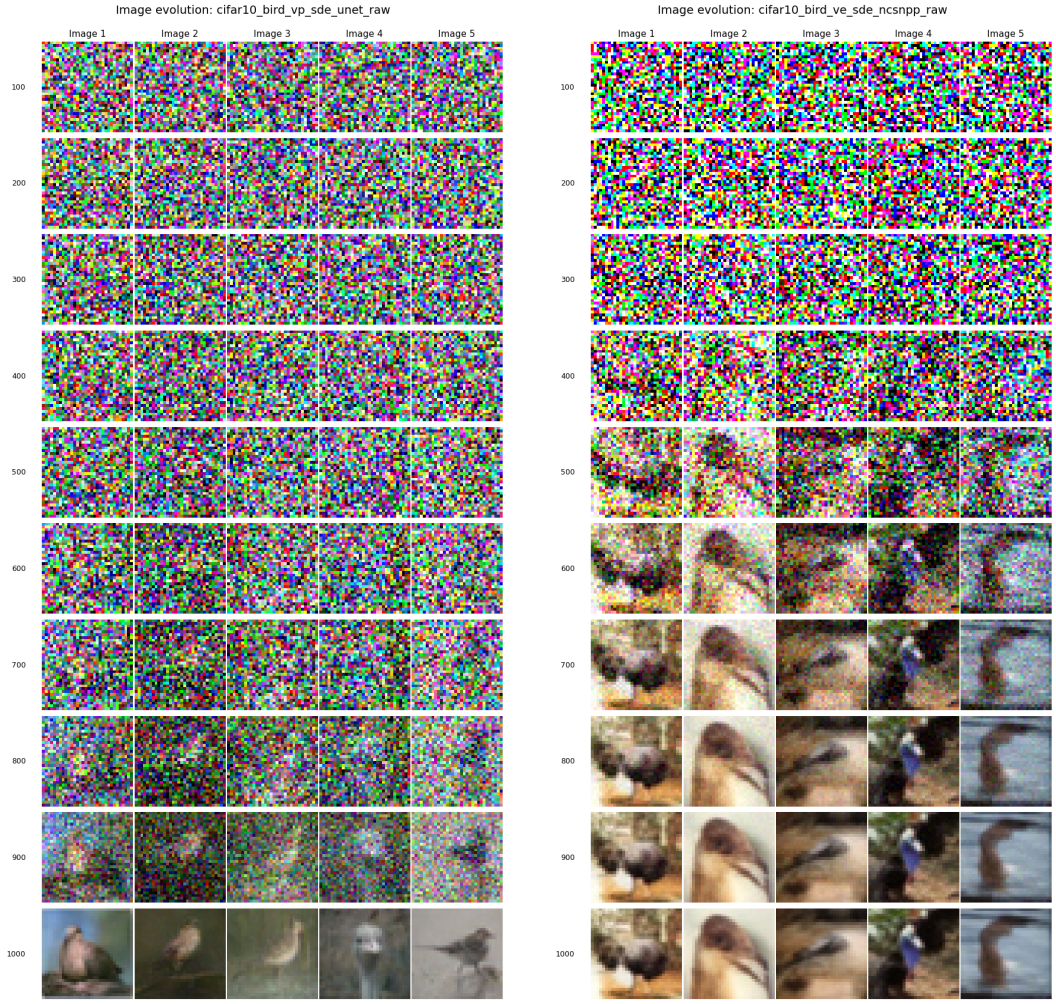


Figure A.2: FID evolution using 5000 generated images for VP U-Net and VE NCSN++ trained on non-augmented data, instead of 1000 as in Table 2.

Sampling-Step Evolution



(a) VP U-Net

(b) VE NCSN++

Figure A.3: Image evolution during sampling for VP U-Net and VE NCSN++ models on CIFAR-10 bird images. The image shows qualitative sample evolution from 100 to 1000 sampling steps.

An interesting observation is that the VE NCSN++ model produces recognizable bird like structure earlier in the sampling procedure than the VP U-Net model. Around step 600, the VE samples are already distinguishable from noise, while the VP samples become visually clear around step 900. This is reasonable because VP-SDE and VE-SDE use different forward noise processes, defined in Eq. 4 and Eq. 5 and therefore equal sampling step indices do not correspond to equal noise levels or signal-to-noise ratios.

Corrector-Step Experiment

Table 4: Effect of corrector steps on FID and nearest-neighbor memorization metrics for the non-augmented VP U-Net model. 1000 generated images were used in the FID calculations

Dataset	Model	SDE	Corrector steps	FID train	FID test	FID gap	Train NN MSE	Test NN MSE	NN gap
CIFAR-10 Bird	U-Net	VP	0	**57.910**	**59.301**	1.391	0.056379	0.069794	0.013415
CIFAR-10 Bird	U-Net	VP	1	59.409	61.199	1.790	0.054426	0.066165	0.011739
CIFAR-10 Bird	U-Net	VP	2	58.920	60.641	1.722	0.053612	0.065503	0.011892
CIFAR-10 Bird	U-Net	VP	3	58.860	60.866	2.006	0.055193	0.066920	**0.011728**
CIFAR-10 Bird	U-Net	VP	4	59.108	60.324	**1.216**	0.054812	0.069601	0.014788

Figure A.4: Generated CIFAR-10 Bird samples from the non-augmented VP U-Net model for different numbers of corrector steps. Columns correspond to corrector steps 0–4, where 0 is sampling without a Langevin corrector. 1000 sampling steps was used.

The corrector-step experiment shows that adding Langevin corrector steps did not improve the non-augmented VP U-Net model. The predictor only sampler, corresponding to 0 corrector steps, achieved the best train and test FID values. Adding 1–4 corrector steps slightly reduced the nearest-neighbor gap for some runs, but this came with worse FID and no clear visual improvement. The generated samples looked similar for every corrector step.

EMA vs Raw Model Comparison

Table 5: Full results on CIFAR-10 Bird. All models use 1000 sampling steps and 1000 generated images for FID and memorization calculations.

Data	Aug.	SDE / Architecture	Weights	FID ↓	Mean	Median	Min	Max
CIFAR-10 Bird	No	VP DDPM++	EMA	60.15	0.0636	0.0606	0.0240	0.1323
CIFAR-10 Bird	No	VP DDPM++	Raw	63.02	0.0689	0.0623	0.0293	0.1771
CIFAR-10 Bird	No	VE NCSN++	EMA	59.07	0.0896	0.0840	0.0175	0.2250
CIFAR-10 Bird	No	VE NCSN++	Raw	63.93	0.0894	0.0839	0.0191	0.1859
CIFAR-10 Bird	No	VP U-Net	EMA	**52.12**	0.0151	0.0124	0.0012	0.0583
CIFAR-10 Bird	No	VP U-Net	Raw	61.14	0.0177	0.0108	0.0012	0.0917
CIFAR-10 Bird	Yes	VE NCSN++	EMA	58.32	0.1978	0.1696	0.0727	0.5222
CIFAR-10 Bird	Yes	VE NCSN++	Raw	68.47	0.1726	0.1463	0.0642	0.4028
CIFAR-10 Bird	Yes	VP DDPM++	Raw	73.37	0.0901	0.0796	0.0190	0.2303
CIFAR-10 Bird	Yes	VP U-Net	EMA	61.10	0.0908	0.0780	0.0264	0.2295
CIFAR-10 Bird	Yes	VP U-Net	Raw	72.15	0.1019	0.0900	0.0230	0.3033

What is interesting is that EMA generally improves FID compared to the raw weights when both versions were evaluated. The best overall FID is obtained by the non-augmented VP-SDE U-Net with EMA weights, reaching an FID of 52.12. However, this model also has the lowest nearest-neighbor distances, suggesting stronger closeness to the training data. The augmented models usually have larger nearest-neighbor distances, indicating weaker memorization, but often worse FID. This supports the main conclusion that augmentation can reduce memorization indicators, but does not necessarily improve sample quality.

Qualitative Samples and Nearest-Neighbor Comparisons

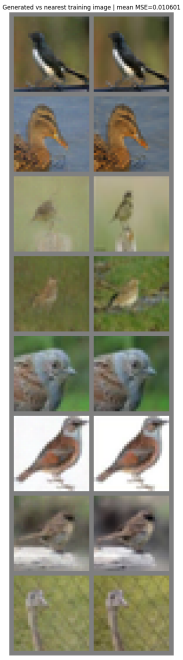
CIFAR-10 Bird

Generated CIFAR-10 Bird samples

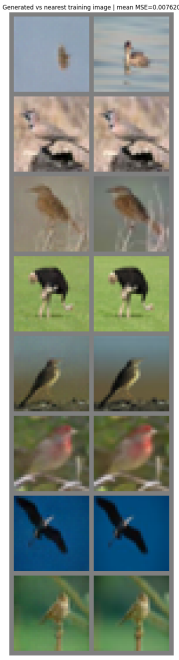
(a) VP DDPM++ EMA

(b) VE NCSN++ EMA

Nearest-neighbor comparison for VP U-Net



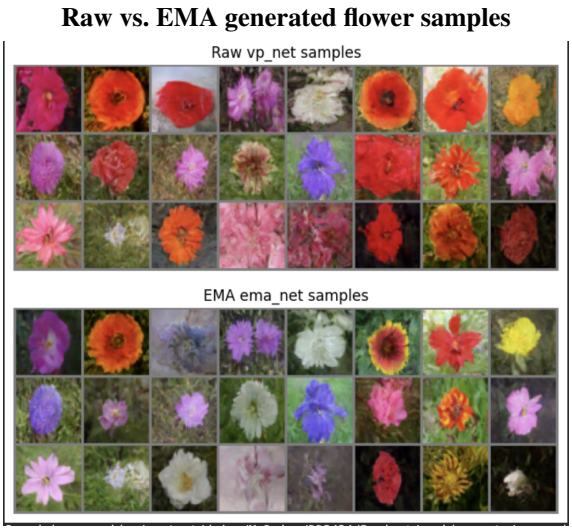
(c) EMA, mean NN MSE $= 0.0106$



(d) Raw, mean NN MSE $= 0.0076$

Figure A.5: Qualitative CIFAR-10 Bird results. The first row shows generated bird samples from two competitive non-augmented EMA models. The second row compares generated VP U-Net samples with their nearest training images, where the small nearest-neighbor distances indicate strong memorization.

Oxford Flowers



(a) EMA, mean NN MSE $= 0.094$ (b) Raw, mean NN MSE $= 0.124$

Figure A.6: Qualitative Oxford Flowers results for the non-augmented VP DDPM++ model. The first row compares raw and EMA generated samples, where EMA produces more coherent flower images. The second row compares generated flowers with their nearest training images; the pairs are similar in color and coarse shape, but usually differ in details.

Sampling Algorithm

Predictor-Corrector Sampling with Euler–Maruyama

Algorithm 1 Predictor-Corrector Sampling

Require: Score network s_θ, grid $1 = t_0 > \cdots > t_N = \varepsilon$, corrector steps M
Require: Drift $f(x,t)$, diffusion $g(t)$, initial distribution p_T

Sample $x \sim p_T$
for $i = 0, \ldots, N-1$ **do**
  $t \leftarrow t_i$, $t' \leftarrow t_{i+1}$, $h \leftarrow t_i - t_{i+1}$
  Sample $z \sim \mathcal{N}(0, I)$
  $s \leftarrow s_\theta(x, t)$
  $x_{\text{mean}} \leftarrow x + \big[-f(x,t) + g(t)^2 s\big]\, h$
  $x \leftarrow x_{\text{mean}} + g(t)\sqrt{h}\, z$
  for $j = 1, \ldots, M$ **do**
    Sample $z \sim \mathcal{N}(0, I)$
    $x \leftarrow x + \eta s_\theta(x, t') + \sqrt{2\eta}\, z$
  end for
end for
return x_{mean}

Perturbation Kernels

Given the linear schedule

$$\beta(t) = \beta_{\min} + t\big(\beta_{\max} - \beta_{\min}\big),$$

the perturbation kernel for the VP SDE can be written as

$$p_{0t}(x(t)) = \mathcal{N}\Big(x(t)\ ;\ \exp\Big(-\tfrac{1}{2}t\beta_{\min} - \tfrac{1}{4}t^2(\beta_{\max} - \beta_{\min})\Big)\, x(0),\ I - \exp\Big(-t\beta_{\min} - \tfrac{1}{2}t^2(\beta_{\max} - \beta_{\min})\Big)\, I\Big). \tag{12}$$

Given the geometric noise schedule

$$\sigma(t) = \sigma_{\min}\left(\frac{\sigma_{\max}}{\sigma_{\min}}\right)^t,$$

the perturbation kernel for the VE SDE is given by

$$p_{0t}(x(t)) = \mathcal{N}\Big(x(t)\ ;\ x(0),\ \sigma_{\min}^2\big(\frac{\sigma_{\max}}{\sigma_{min}}\big)^{2t}, I\Big). \tag{13}$$

Model Architecture Details

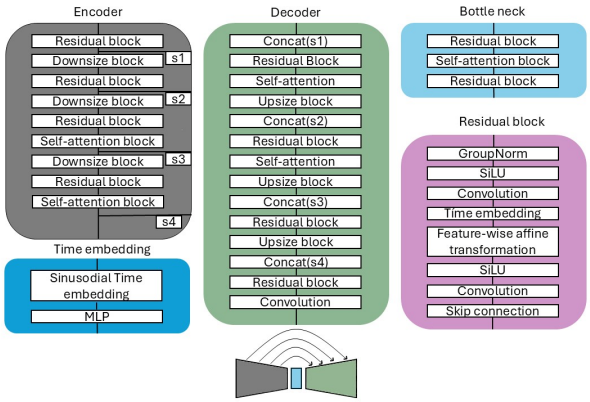


Figure A.7: Overview of the U-Net architecture. The encoder applies residual blocks and downsampling, the bottleneck includes self-attention, and the decoder upsamples with skip connections. Time information is injected through sinusoidal embeddings and feature-wise affine transformations in the residual blocks.

Table 6: Architecture configurations used in the experiments.

Dataset / model	Main channels	Blocks / attention	Conditioning and other settings
CIFAR-10 Bird, VE-SDE, NCSN++	$n_f = 128$, channel multipliers $(1, 2, 2, 2)$	4 residual blocks per resolution; attention at 16×16	Fourier embedding, scale 16; BigGAN residual blocks; FIR; skip rescale; dropout 0.1; $\sigma_{\min} = 0.01$, $\sigma_{\max} = 50$
CIFAR-10 Bird, VP-SDE, DDPM++	$n_f = 64$, channel multipliers $(1, 2, 2, 2)$	2 residual blocks per resolution; attention at 16×16	Positional embedding; BigGAN residual blocks; FIR; skip rescale; dropout 0.1; $\beta_{\min} = 0.1$, $\beta_{\max} = 20$
Oxford Flowers, VP-SDE, DDPM++	$n_f = 64$, channel multipliers $(1, 2, 2, 2)$	2 residual blocks per resolution; attention at 16×16	Same as CIFAR-10 Bird DDPM++, but with 48×48 input images
CIFAR-10 Bird, VP-SDE, U-Net	Base channels $= 128$	Attention at 16×16, 8×8, and bottleneck	Sinusoidal time embedding; GroupNorm; SiLU; encoder–decoder U-Net with skip connections